

CoderCommands MCP System

Step 07 Record — Docker Compose Wiring

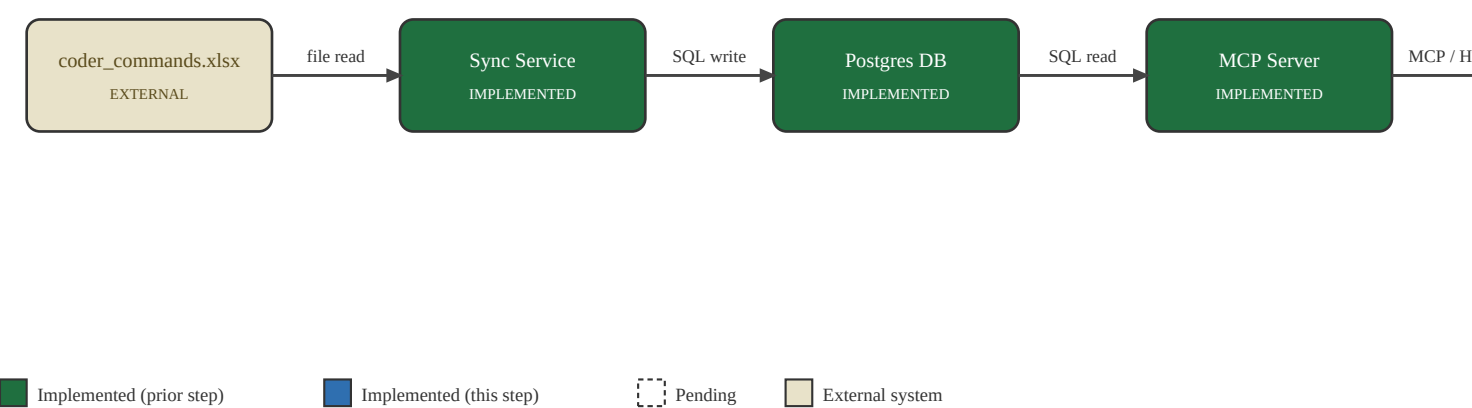
DOCUMENT	CoderCommands MCP System — Step Record
SYSTEM	CoderCommands MCP Command-Reference Server
STEP	07 — Docker Compose Wiring
DATE	2026-08-15
REVISION	1.0
STATUS	Complete

1. Purpose & Scope

Replace the ad hoc host-published-port test setup used to validate Steps 4 through 6 individually with the real deployment topology: four services on a private internal network, addressed by service name, with only the FastAPI gateway exposed to the host. This step is where the system context diagram stops being a design intent and becomes an actual running network.

2. System Context Diagram

Signal-flow view of the four functional units (Sync Service, Postgres, MCP Server, FastAPI Gateway) plus the two external endpoints (source workbook, Claude Code client). Fill/border indicates build state as of this step.



3. File / Component Register

Role of every file or component introduced or modified in this step, with its upstream and downstream interfaces (I/O-list style).

Ref	File / Component	Role	Interface In	Interface Out
R1	<code>docker-compose.yml</code>	Defines all four services, an internal bridge network (codercommands-net), a named volume for Postgres data, and healthcheck-gated startup ordering.	—	—

Ref	File / Component	Role	Interface In	Interface Out
R2	.env.example / .env	Runtime configuration: DB credentials, xlsx host path, sync poll interval, gateway host port. .env is gitignored; removed two unused variables (POSTGRES_PORT, MCP_SERVER_PORT) left over from Step 1 scaffolding once it was clear neither service needed a configurable port.	—	All services

4. Work Performed

- postgres: no published host port (this machine already runs an unrelated Postgres container on the default 5432); healthcheck via pg_isready gates every dependent service's startup.
- sync: depends_on postgres with condition: service_healthy; xlsx bind-mounted read-only from the path in .env.
- mcp-server: same healthy-postgres dependency; no published port -- reachable only from other containers on codercommands-net.
- fastapi: the only service with a published port (host FASTAPI_HOST_PORT → container 8100); own healthcheck via its /healthz endpoint.
- Brought the full stack up with `docker compose up -d --build` and validated the real, networked path end-to-end -- not the host-port workarounds used in Steps 4–6.

5. Issues Encountered & Resolution

Issue	Root Cause	Resolution
`docker logs codercommands-sync` showed no output at all, even though the sync had already succeeded (confirmed 608 rows and a 'success' row in sync_runs by querying Postgres directly).	Python fully buffers stdout when it isn't attached to a TTY, which is exactly the container's situation. watcher.py's print() calls were sitting in a buffer that hadn't flushed, so the container looked silent/dead from the outside despite working correctly.	Added ENV PYTHONUNBUFFERED=1 to sync/Dockerfile. Rebuilt and confirmed both startup and per-sync log lines now appear immediately in `docker logs`.

Issue	Root Cause	Resolution
Every tool call through the FastAPI gateway failed once the stack ran on its real compose network, with mcp-server logging "Invalid Host header: mcp-server:9000" and returning 421 Misdirected Request -- despite the identical proxy code having worked in Step 6.	The MCP SDK enables DNS-rebinding protection on the Streamable HTTP transport by default, allow-listing only Host: localhost/127.0.0.1. Step 6's test happened to pass this check by coincidence: it reached the container via its published port at 127.0.0.1, which matched the default allow-list. Reaching it by its real service name (mcp-server:9000, as the gateway does in the actual compose network) does not, so the vulnerability only surfaced once the real network topology was exercised for the first time.	Passed an explicit TransportSecuritySettings to streamable_http_app() allow-listing mcp-server:9000 (plus localhost/127.0.0.1 for direct dev use) rather than disabling the protection outright -- kept in place since it's a real, if narrow, security control, and the allow-list matches this container's actual reachability (no published port, only callable from other containers on the compose network).
Docker Desktop returned a transient "error during connect ... EOF" on the named pipe during `docker compose up -d --build`, partway through container creation.	Recurring Docker Desktop-on-Windows quirk observed several times across this project (also hit in Steps 2 and 4) -- not specific to this compose file.	Re-ran the same command; compose resumed from where it left off and created the remaining containers normally. Not something to design around, just a known hiccup worth recognizing if it recurs during Step 8 or later operation.

6. Verification Record

Check	Method	Result
docker compose config resolves cleanly, including Windows bind-mount paths	docker compose config	PASS
All four containers reach a running/healthy state with correct dependency ordering	docker compose ps -- postgres healthy before sync/mcp-server start	PASS
sync populates Postgres on the real compose network (no host-published DB port)	608 rows / 14 topics, confirmed via psql inside the postgres container	PASS
Sync log output visible in real time after the buffering fix	docker logs codercommands-sync shows startup + per-run lines immediately	PASS
Full MCP path works end-to-end through the published gateway port only	MCP client → 127.0.0.1:8100/mcp → fastapi → mcp-server (by service name) → postgres: 16 tools, correct search result	PASS

7. Next Step / Open Dependencies

Step 8 is a dedicated, broader verification pass against this running stack -- including confirming a live xlsx edit propagates through the watcher without any manual step, which hasn't yet been exercised against the real

compose deployment.